

Google Cloud Platform + AI Fundamentals

Universidad Austral
Facultad de Ingeniería
Primer Cuatrimestre 2026

Profesores
Mora Villa Abrille (BVilla@austral.edu.ar)
Rodrigo Pazos (RPazos@austral.edu.ar)

Reliability

- *Reliability is the most fundamental feature of any product. If a person can't trust a service, the service has failed.*
- *A systems ability to consistently perform its intended function under varying conditions over a specified period of time*

Un repaso rápido

- Hace algunos años las empresas tenían un perfil muy común: el **sysadmin**
- Esta persona o grupo de personas administraba los sistemas en producción de manera **manual o semiautomática**, realizando tareas de mantenimiento, actualización y despliegue del software generado por el equipo de desarrollo.
- Un problema obvio de esto es que conforme nuestros sistemas crecen y se complejizan **necesitamos más y más gente que administre el sistema.**
- Un problema no tan obvio es que existen dos grupos de personas que están en **constante tensión**: los desarrolladores y los sysadmin. Unos quieren llevar sus cambios a producción mientras que los otros quieren minimizar la cantidad de cambios, porque el cambio lleva a errores.

Site Reliability Engineering (SRE)

- **Ben Treynor Sloss** crea en 2003 el concepto de SRE, cuando le asignan que lidere un equipo de sysadmins en Google. Por 13 años desarrollaron métodos para reducir la tensión entre los objetivos de los dos equipos (dev y sysadmin) tratando de automatizar y reducir la fricción en el mantenimiento y despliegue de los sistemas
- En 2016 Google publica un libro donde explican sus prácticas en detalle. Se adoptan muchos conceptos en empresas de software de todo tipo y tamaño.



Availability	Downtime / Year	Downtime / Month	Downtime / Week	Downtime / Day
99.999%	5.256 Minutes	0.438 Minutes	0.101 Minutes	0.014 Minutes
99.995%	26.28 Minutes	2.19 Minutes	0.505 Minutes	0.072 Minutes
99.990%	52.56 Minutes	4.38 Minutes	1.011 Minutes	0.144 Minutes
99.950%	4.38 Hours	21.9 Minutes	5.054 Minutes	0.72 Minutes
99.900%	8.76 Hours	43.8 Minutes	10.108 Minutes	1.44 Minutes
99.500%	43.8 Hours	3.65 Hours	50.538 Minutes	7.2 Minutes
99.250%	65.7 Hours	5.475 Hours	75.808 Minutes	10.8 Minutes
99.000%	87.6 Hours	7.3 Hours	101.077 Minutes	14.4 Minutes

Site Reliability Engineering (SRE)

- **La confiabilidad extrema es costosa.** Ya sea porque tenemos muchísimo costo en hardware redundante o porque dedicamos esfuerzos de ingeniería a mejorar la confiabilidad en vez de desarrollar nuevas features para el sistema
- **La diferencia entre 99.9% de disponibilidad y 99.99% requiere muchísimo esfuerzo** y muchas veces **la mejora percibida por los usuarios es mínima o nula**, porque otra parte del sistema es menos confiable (GCS es mucho más confiable que la conexión de red de celular)
- Entonces es necesario construir un consenso entre el equipo de producto y el equipo de ingeniería para poder elegir cuales son los objetivos de confiabilidad. Tenemos que determinar la **tolerancia al riesgo** que puede manejar el producto

SLI, SLO y SLA

- **Service Level Indicator:** Lo primero que nos tenemos que preguntar es que vamos a medir. Existen infinitas variables y aspectos de un sistema que podemos medir. Una cantidad reducida de esas variables van a servirnos para entender que tan bien funciona el sistema. (Que significa bien?)
- **Service Level Objective:** Una vez que podemos medir algo podemos establecer objetivos. Estos objetivos sirven de umbrales: si estamos por debajo de este umbral tienen que sonar alarmas que nos hagan poner foco en mejorar la confiabilidad del sistema y no sacar nueva funcionalidad
- **Service Level Agreement:** acuerdos legales con clientes que pueden tener implicancias financieras. El SLA tiene que ser más relajado que el SLO.

Error Budget

- El equipo de Treyner Sloss crea este concepto para poder determinar de **manera objetiva** si el equipo de desarrollo tiene que trabajar en **confiabilidad o en features nuevas**.
- Se mide en una rolling window de X días para atrás el conjunto de métricas y se lo compara con el SLO. Si el resultado da que el equipo está por debajo del SLO tiene que trabajar en mejorar la confiabilidad. En caso contrario puede seguir trabajando en nuevas features.
- Siempre y cuando exista budget hay que usarlo para sacar nueva funcionalidad que debería mejorar la experiencia de los clientes o atraer nuevos usuarios.

Toil

- *Trabajo asociado con correr un servicio en producción que suele ser manual repetitivo, automatizable, táctico, que no ofrece valor que perdura en el tiempo y que escala de manera lineal.*
- **Táctico:** reactivo, que se ejecuta ante una interrupción o alerta. No es estratégico y proactivo.
- **Valor perdurable:** un cambio o mejora en un proceso que es repetible y reduce fricción o costos. Automatizar algún paso del despliegue por ejemplo
- La idea es reducir este trabajo porque se le suele asignar a algunas personas del equipo que dedican tiempo en algo que no suma tanto valor, además de que suele ser desgastante por el estrés de generar un error

Four Golden Signals

- **Latencia:** el tiempo en procesar una request. Es necesario diferenciar la latencia entre errores y respuestas exitosas.
- **Tráfico:** la demanda que está soportando el sistema. Se puede medir en request por segundo, conexiones concurrentes o lecturas a disco, dependiendo el tipo de sistema
- **Errores:** pueden ser explícitos, implícitos o que fallen a algún requerimiento no funcional.
- **Saturación:** el uso de los recursos del sistema. Lo mejor es medir el recurso por el que nuestro sistema está limitado (memoria, disco, CPU, etc). La saturación afecta rápidamente a la latencia

MTTR vs. MTBF

- **Mean Time to Repair** es el promedio de tiempo que tardamos desde que encontramos un bug en producción a que lo solucionamos. Este valor puede ser cero siempre y cuando podamos capturar los bugs en alguna etapa de testing previo (o no cometer nunca ningún error). Si el error ya está en producción un canary release y una herramienta de rollback pueden ayudar a identificar y mitigar el impacto de los errores
- **Mean Time Between Failures** es el tiempo promedio que pasa entre dos errores en nuestro sistema. Es imposible que sea infinito pero tenemos que apuntar a que sea lo más alto posible con mecanismos que identifiquen los bugs lo antes posible.

CI/CD Pipelines